



Proyecto 1

Computabilidad y Complejidad – CI0124

Prof. Maureen Murillo R.
II-2026

Título: Rutas en Internet

Descripción del problema

Imaginá Internet no como una sola red masiva, sino como una malla interconectada de miles de redes independientes —desde proveedores de telecomunicaciones y gigantes tecnológicos hasta universidades y centros de datos— que necesitan ponerse de acuerdo para intercambiar información. El protocolo que hace posible esta colaboración a escala planetaria es **BGP (Border Gateway Protocol)**, el «sistema de navegación» invisible del mundo digital. Cada vez que enviás un mensaje, ves un video o accedés a un servicio en la nube, BGP determina en fracciones de segundo la ruta exacta que recorrerán tus datos a través de decenas de países.

Para comprender esta dinámica es necesario introducir tres conceptos centrales de BGP. Un **sistema autónomo (AS)** es una red o conjunto de redes bajo una administración única —un proveedor de Internet, una universidad, una empresa— identificada por un número (ASN); Internet es, en el fondo, una de miles de estos ASes negociando entre sí. Un **prefijo** es un bloque de direcciones IP (por ejemplo, 192.0.2.0/24) que representa un destino alcanzable: cuando un AS "anuncia" un prefijo, está declarando que conoce una ruta hacia esas direcciones. Finalmente, un **peer** es el AS vecino con el que un router intercambia directamente información de rutas mediante una sesión BGP; no debe confundirse con el prefijo, ya que el peer no es un destino sino la fuente inmediata de la información de una ruta. Cada anuncio de ruta, entonces, indica el prefijo de destino, el AS Path (la secuencia de sistemas autónomos que la ruta atraviesa) y el peer a través del cual se recibió esa información.

Investigar BGP es vital porque el protocolo fue diseñado en los inicios de Internet bajo un principio de confianza implícita, lo que lo vuelve vulnerable a errores de configuración, fallas de enrutamiento y ciberataques que pueden desconectar regiones enteras o desviar tráfico sensible. Estudiar BGP es, en esencia, aprender a diagnosticar y proteger la infraestructura crítica que sostiene la sociedad moderna. Para analizar la salud de Internet y detectar estos problemas, los observatorios globales de red capturan constantemente la actividad de BGP y la exportan en registros estandarizados conocidos como **dumps de MRT (Multi-Threaded Routing Toolkit)**. Al trabajar con estos archivos en formato de texto, nos encontramos con tablas masivas donde cada línea representa una ruta específica expresada mediante campos delimitados por caracteres especiales. Un ejemplo de una línea de estos archivos es:

```
TABLE_DUMP2|1785888000|B|177.101.16.80|53046|7.0.0.0/8|53046 61626 14840 3356 749
```

Crear un **parser para estos archivos de MRT** es el primer paso fundamental para procesar este volumen de datos, extraer atributos clave y transformarlos en estructuras de datos organizadas en memoria. Al construir su propio parser, no solo aprenderán a interpretar la anatomía real de las rutas de Internet, sino que desarrollarán la herramienta base necesaria para generar mapas topológicos, analizar el comportamiento de la red y alimentar modelos de análisis de datos e inteligencia artificial capaces de detectar anomalías en tiempo real.



Objetivo

Este proyecto consiste en crear un analizador sintáctico (*parser*) que valide la correctitud del contenido de archivos dump de MRT. El *parser* debe ser parte de una aplicación que, utilizando los datos de entrada, permita a los usuarios obtener en forma amigable información relevante.

El proyecto deberán desarrollarlo en Python utilizando la biblioteca PLY (Python Lex-Yacc).

Tareas específicas

El funcionamiento de la aplicación se divide en varios componentes.

1. Entrada

La aplicación deberá leer un archivo de texto con el dump de MRT. Con este proyecto se le proporcionan algunos archivos de ejemplo. Su aplicación debe tomar en cuenta que el archivo puede venir con errores y la aplicación debe reportar cualquier problema que encuentre en la entrada.

2. Analizador léxico

El analizador léxico (*lexer*) verifica que el archivo contenga piezas de información válidas (tokens) con un formato correcto. No revisa si la estructura del archivo está bien, solo si contiene tokens válidos. Un ejemplo de una línea de un archivo es:

```
TABLE_DUMP2|1785888000|B|177.101.16.80|53046|7.0.0.0/8|53046 61626 14840 3356 749
```

Cada línea del archivo está compuesta de los siguientes campos.

1. **Record type:** tiene que ser TABLE_DUMP2
2. **Timestamp:** numérico. No debería permitirse cualquier número. Revisar características en <https://datatracker.ietf.org/doc/html/rfc6396>
3. **State:** B, A o W
4. **Peer IP:** una dirección IP
5. **Peer AS:** numérico. Investigar en Internet cuál es el valor máximo.
6. **Destination prefix:** una dirección IP, seguida de un "/", seguida de un número entre 1 y 31. No es obligatorio manejar la relación entre la dirección IP y la longitud de la máscara.
7. **AS Path:** una lista de identificadores de sistemas autónomos, la cual contiene al menos uno. En este *path*, el primer identificador de AS debe ser el mismo que en el campo 5. Todo *path* comienza con el AS propio.

3. Analizador sintáctico

El analizador sintáctico (*parser*), utilizando como insumo el resultado generado por el *lexer*, verifica que la estructura del archivo sea la correcta. Por ejemplo, debe verificar que cada línea contiene todos los campos en el orden correcto, que el campo 7 es una lista de mínimo un sistema autónomo, el cual debe corresponder al reportado en el campo 5, entre otros.



4. Aplicación

Una vez que el *parser* concluyó exitosamente, la aplicación deberá:

1. Reportar si hay más de dos sistemas autónomos reportando un prefijo como propio. Si encontraran, por ejemplo, estas tres líneas para el mismo prefijo 8.8.8.0/24:

AS Path: [3549, 174, 15169]

AS Path: [6939, 15169]

AS Path: [7018, 3257, 64512] ← este último AS es distinto

Los dos primeros coinciden en que el origen es 15169 (no hay conflicto, es normal que varios peers reporten el mismo origen por caminos distintos). Pero el tercero tiene un origen diferente (64512) para el mismo prefijo — eso sí sería sospechoso, y es justo el tipo de anomalía (posible *prefix hijacking*) que esta funcionalidad busca detectar.

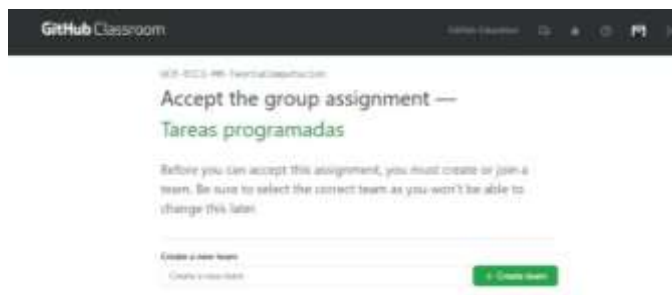
2. Dado un número de AS, mostrar de forma tabulada todas sus aristas (cada 4to campo indica la dirección IP de una arista de un AS) y todos los prefijos conocidos por el AS a través de esa arista. Para cada prefijo, mostrar también el AS Path.
3. Dado un prefijo A, (que será digitado de la misma forma que en el campo 6) y un prefijo B (que también será digitado de la misma forma que en el campo 6), muestre todas las rutas disponibles entre A y B. No se debe encontrar la ruta más corta, simplemente todas las rutas, incluyendo dos o más rutas desde un mismo AS. Para esta funcionalidad, se espera un diseño inteligente de su parser y sus acciones, para evitar después una implementación que recorra toda la estructura de datos buscando información.



Entregables

Este proyecto debe ser resuelto en grupos de 3 personas. Cada grupo deberá crear un repositorio en Github a través del siguiente enlace <https://classroom.github.com/a/WuRuvwKA>

El primer estudiante de cada equipo que accese este enlace será el que creará el repositorio y deberá usar el nombre del equipo que previamente se asignaron.



Cuando los demás miembros del equipo accesen el enlace anterior deberán unirse al repositorio que ya habrá creado el primer compañero. Una vez creado el repositorio, deben crear una carpeta para cada proyecto, y dentro de la carpeta para el proyecto 1 deben crear una subcarpeta para cada entregable, dentro de la cual quedarán los documentos que se revisarán.

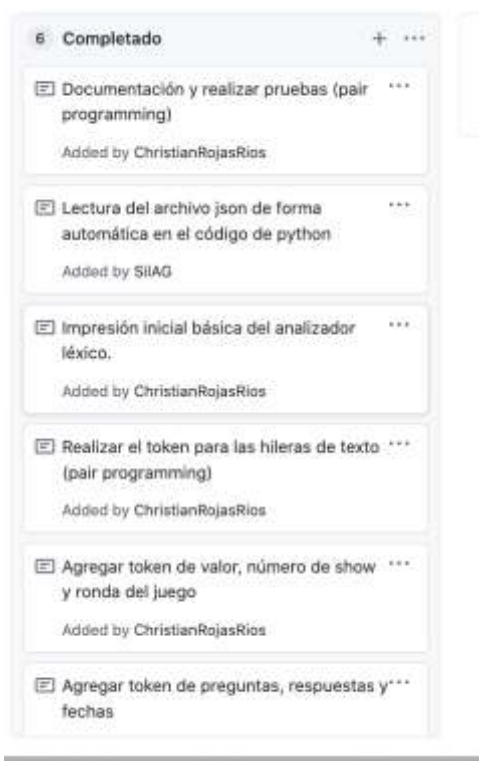
Cada equipo deberá presentar los siguientes entregables:

1. (10%) En el primer avance debe presentar un documento pdf con lo siguiente:
 - a) Pídanle a alguna herramienta de IA que les genere un diagrama de los conceptos involucrados en este proyecto y la explicación que ustedes consideren necesaria. Preséntenlo como parte de este entregable e indiquen cuál herramienta se los generó. Durante la exposición deben explicarlo.
 - b) Explicación de las estructuras de datos finales más adecuadas en donde los datos del archivo deberían quedar almacenados, de tal manera que faciliten el logro eficiente de las funcionalidades solicitadas. Deben justificar su escogencia.Para este entregable NO se requieren múltiples *commits* de todos los miembros del equipo pero sí debe incluir la planificación con *Projects* (revisar siguiente sección). Todos los miembros del grupo deberán exponer. **Fecha: Jueves 20 de agosto, 7:00am.**
2. (20%) Analizador léxico funcionando: en esta entrega la herramienta debe ser capaz de reconocer todos los tokens necesarios para procesar el archivo de entrada. **Fecha: Lunes 24 de agosto, 7:00am.**
3. (35%) Analizador sintáctico revisando errores pero sin crear objetos o estructuras de datos dinámicamente: en esta entrega se revisa que el archivo esté sintácticamente correcto y se imprime error en caso contrario. **Lunes 14 de setiembre, 7:00am.**
4. (15%) Creación dinámica de los objetos o las estructuras de datos: a la entrega anterior, se le agrega la capacidad de crear dinámicamente los objetos o estructuras necesarias para la aplicación. **Lunes 21 de setiembre, 7:00am.**
5. (20%) Implementación de la aplicación completa: entrega final con toda la funcionalidad. **Lunes 5 de octubre, 7:00am.**



Recomendaciones de trabajo

1. GitHub es un repositorio con control de versiones que permite gestionar proyectos especialmente en casos de trabajo en equipo. El objetivo de uso en este curso no es solamente para realizar las entregas de cada etapa, sino que además se espera que sea utilizado para dar seguimiento al desarrollo de los proyectos, tanto por parte de la docente, del asistente como también de parte de los mismos integrantes de cada equipo. En este sentido, es necesario que para cada etapa quede evidenciado el desarrollo y avance paulatino del trabajo (no solo la entrega final) mediante la planificación y asignación de responsables de cada actividad y la realización de *commits* regulares durante todo el proceso.
2. Tal como se indica en la siguiente sección, debe utilizarse la herramienta *Projects* de la plataforma GitHub para organizar y realizar la planificación detallada de las actividades de cada etapa. Se debe primero crear *issues* y luego utilizarlos en un tablero de *Projects*. El siguiente es un buen video para comprender mejor cómo funcionan estas herramientas, aunque no es necesario que apliquen todas las opciones que brindan: https://www.youtube.com/watch?v=m1_07tIBCy4. La planificación debe ser detallada con actividades específicas, no solo generales. El siguiente ejemplo muestra una buena subdivisión del trabajo:





Aspectos evaluativos

1. La planificación **detallada** del trabajo del equipo (utilizando *Projects*), organizada con actividades específicas y no generales, e indicándose el responsable para cada una, debe realizarse para cada etapa. Tendrá un valor de **2 puntos en cada entregable**.
2. En el repositorio debe haber evidencia del **trabajo individual** de cada uno de los miembros del equipo, mediante la ejecución de *commits* **regulares** (no solo uno o dos *commits* por estudiante) que muestren **aportes significativos e incrementales** del proyecto. Estos *commits* deben realizarse con regularidad entre la fecha del entregable anterior y la próxima, es decir, no deben realizarse exclusivamente en los dos días anteriores a la fecha límite. Aquel estudiante que no haya realizado *commits* antes de la fecha y hora de entrega límite en las **condiciones mencionadas**, se considerará como que no trabajó en el proyecto y perderá los puntos del entregable. Deben respetar la fecha y hora límite de entrega indicada en el enunciado del proyecto.
3. Se espera que todo el proyecto sea desarrollado y escrito desde cero. En este proyecto está prohibido apoyarse en herramientas generadoras de código y lenguaje. En caso de duda, la persona estudiante debe comunicarse con anticipación con la docente y aclarar aspectos del enunciado y evaluación acerca de usos permitidos de ayudas externas, tales como bibliotecas de código. En cualquier evaluación la profesora podrá realizar una entrevista a cualquier persona estudiante para verificar si es capaz de explicar con propiedad la solución entregada. Cuando la persona estudiante no sea capaz de explicar con fluidez su solución durante la exposición o durante la entrevista, el trabajo se considerará como inválido y tendrá un cero de calificación.
4. Todos los integrantes del equipo deben participar en la exposición oral de cada etapa y deberán responder las preguntas de la profesora. Esta exposición oral tiene un valor de **5 puntos en cada entregable**.
5. En la carpeta de cada entregable debe haber un archivo *readme* que explique claramente cómo ejecutar el programa, incluyendo los paquetes que se requiere tener instalados. Este archivo dentro de la carpeta tendrá un valor de **1 punto en cada entregable**.